Experiment 2: Loan Amount Prediction using Linear Regression

Subject: UCS2612 – Machine Learning Algorithms Laboratory

Name : Harishkanna R

Sec : CSE-A

Reg : 3122235001302

Linear Regression for Loan Amount Prediction

1) Aim

To build and evaluate a Linear Regression model for predicting the loan amount sanctioned to users using historical loan data, and to analyze model performance through evaluation metrics and visualizations.

2) Libraries Used

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

3)Objective

The primary objective of this project is to develop a data-driven model that predicts the eligible loan amount for an applicant using Linear Regression, based on key financial and demographic attributes.

4) Mathematical Description

Linear Regression Equation

General form:

$y = b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n$

Where:

- $y \rightarrow$ predicted output (Loan Amount)
- $b_0 \rightarrow$ intercept (bias)
- $b_1, b_2, \ldots, b_n \rightarrow$ coefficients
- $x_1, x_2, \ldots, x_n \rightarrow$ input features

Simple Linear Regression (y = mx + b)

$y = mx + b$

Where:

- $y \rightarrow$ predicted value
- $m \rightarrow$ slope (rate of change)
- $x \rightarrow$ input variable
- $b \rightarrow$ intercept

5) Code with plot

Data Loading

In [2]:
```
df = pd.read_csv("loan_train.csv")
df.head()
```

Out[2]:

	Customer ID	Name	Gender	Age	Income (USD)	Income Stability	Profession	Type of Employment	Locatio
0	C-36995	Frederica Shealy	F	56	1933.05	Low	Working	Sales staff	Sem Urba
1	C-33999	America Calderone	M	32	4952.91	Low	Working	NaN	Sem Urba
2	C-3770	Rosetta Verne	F	65	988.19	High	Pensioner	NaN	Sem Urba
3	C-26480	Zoe Chitty	F	65	NaN	High	Pensioner	NaN	Rura
4	C-23459	Afton Venema	F	31	2614.77	Low	Working	High skill tech staff	Sem Urba

5 rows × 24 columns

In [3]:
```
df.shape
```

Out[3]:
```
(30000, 24)
```

Data Preprocessing

In [4]:
```
df.isnull().sum()
```

Out[4]:
```
Customer ID                       0
Name                              0
Gender                           53
Age                               0
Income (USD)                   4576
Income Stability               1683
Profession                        0
Type of Employment             7270
Location                          0
Loan Amount Request (USD)         0
Current Loan Expenses (USD)     172
Expense Type 1                    0
Expense Type 2                    0
Dependents                     2493
Credit Score                   1703
No. of Defaults                   0
Has Active Credit Card         1566
Property ID                       0
Property Age                   4850
Property Type                     0
Property Location               356
Co-Applicant                      0
Property Price                    0
Loan Sanction Amount (USD)      340
dtype: int64
```

In [5]:
```
df=df.drop(columns=['Customer ID', 'Name', 'Gender', 'Age', 'Income (USD)',
       'Income Stability', 'Profession', 'Type of Employment', 'Location',
       'Expense Type 1', 'Expense Type 2', 'Dependents',
       'No. of Defaults', 'Has Active Credit Card', 'Property ID',
       'Property Age', 'Property Type', 'Property Location', 'Co-Applicant'],axis=1
```

In [6]:
```
df.fillna(df.mean(numeric_only=True),inplace=True)
for col in df.select_dtypes(include="object"):
    df[col].fillna(df[col].mode()[0])
```

In [7]:
```
# cat_cols = df.select_dtypes(include='object').columns
# df[cat_cols].nunique().sort_values(ascending=False)
```

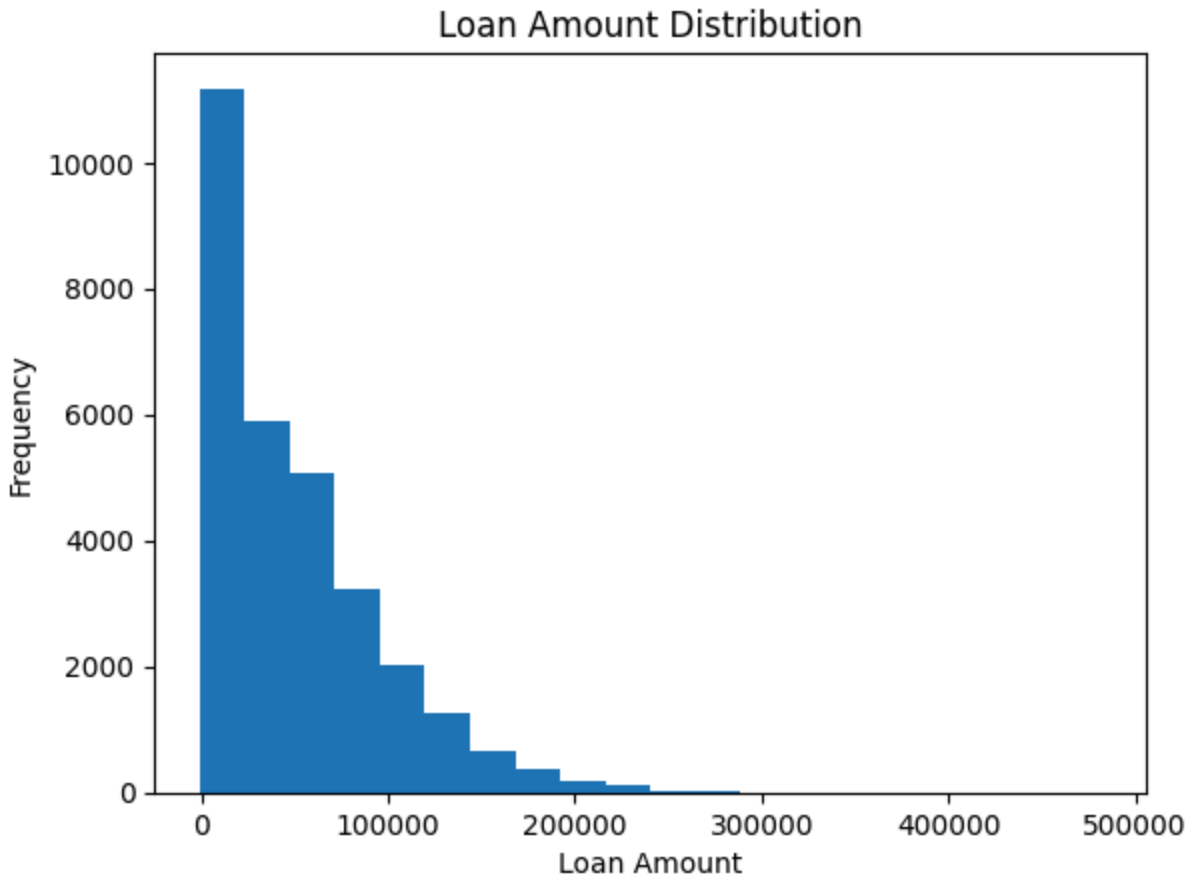
In [8]:
```
df.columns
```

Out[8]:
```
Index(['Loan Amount Request (USD)', 'Current Loan Expenses (USD)',
       'Credit Score', 'Property Price', 'Loan Sanction Amount (USD)'],
      dtype='object')
```

In [9]:
```
scaler = StandardScaler()

X = df[["Current Loan Expenses (USD)","Loan Amount Request (USD)","Credit Score","P
y = df["Loan Sanction Amount (USD)"]

X_scaled = scaler.fit_transform(X)
```

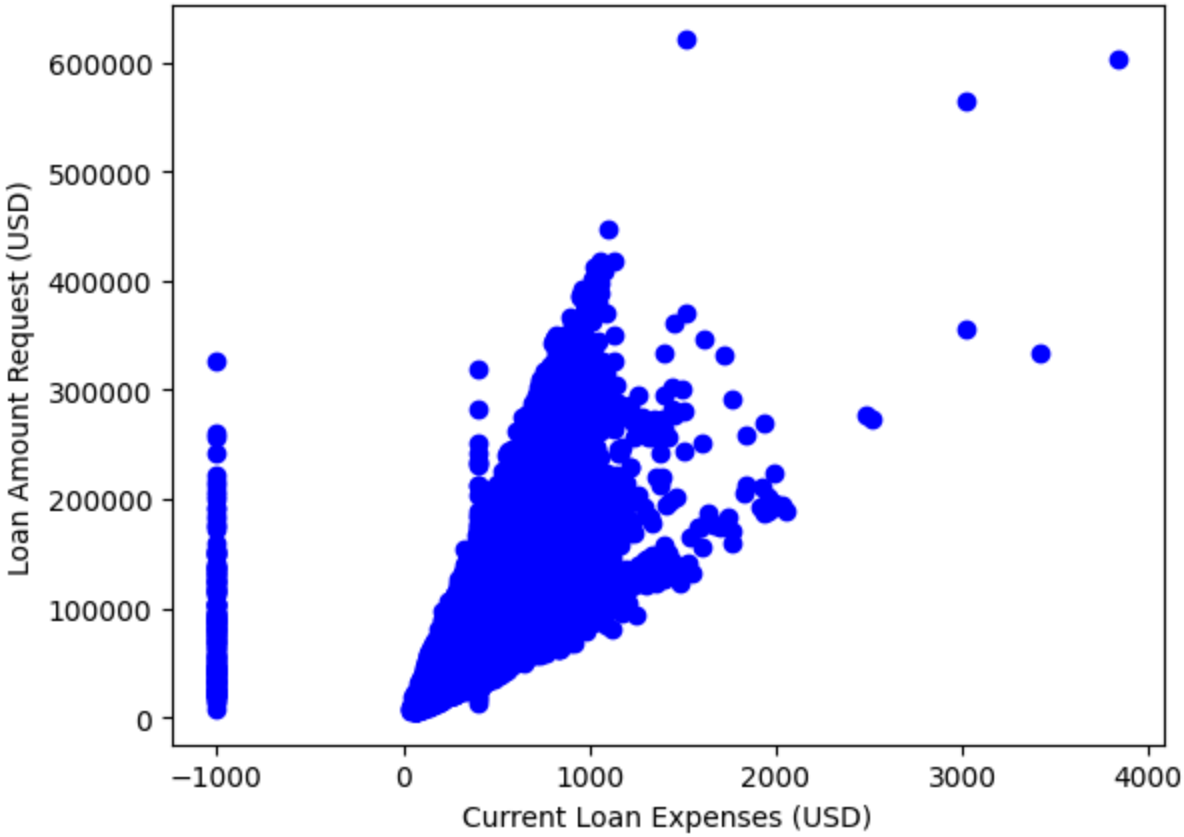
Exploratory Data Analysis

In [10]:
```
plt.hist(y,bins=20)
plt.title("Loan Amount Distribution")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")
plt.show()
```

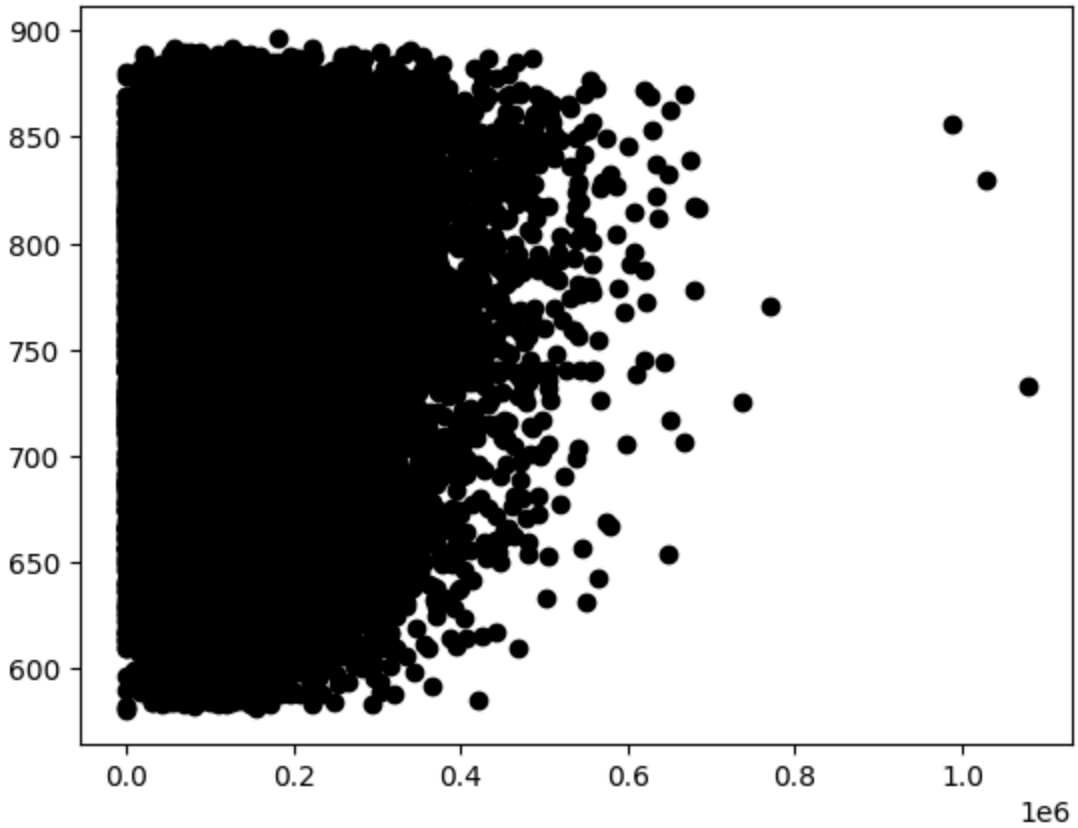


Scatter Plot

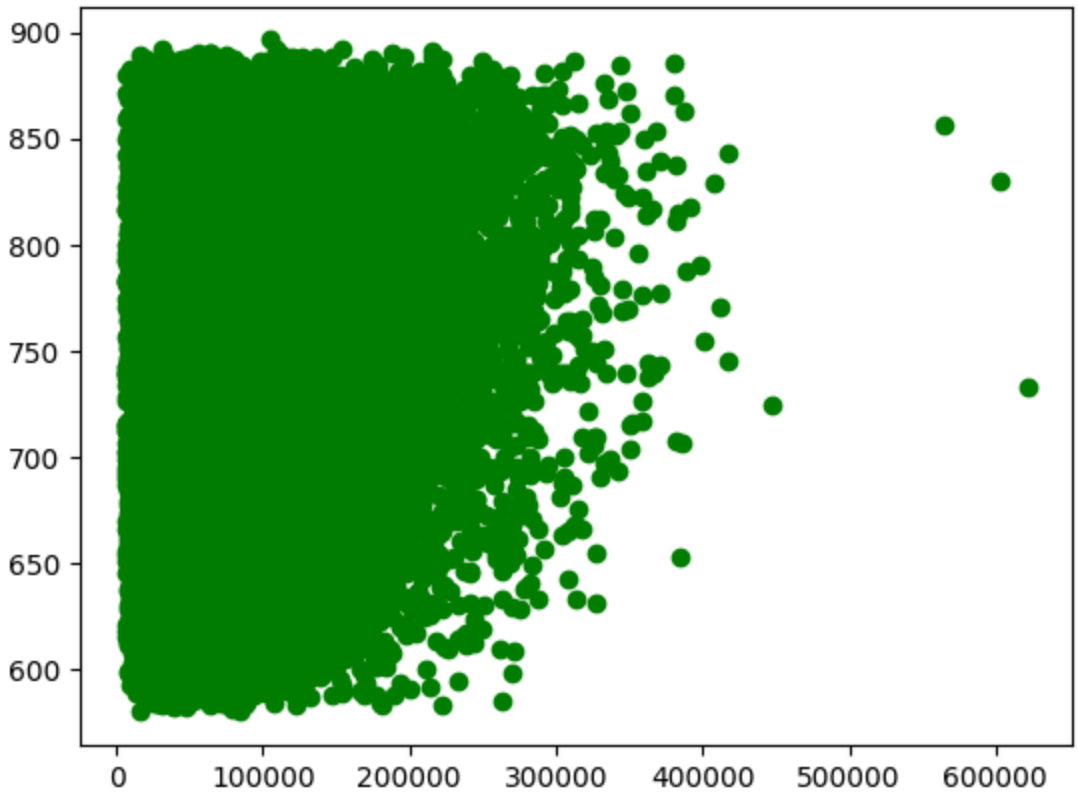
In [11]:
```
plt.scatter(df['Current Loan Expenses (USD)'],df['Loan Amount Request (USD)'],color
plt.xlabel("Current Loan Expenses (USD)")
plt.ylabel("Loan Amount Request (USD)")
plt.show()
```



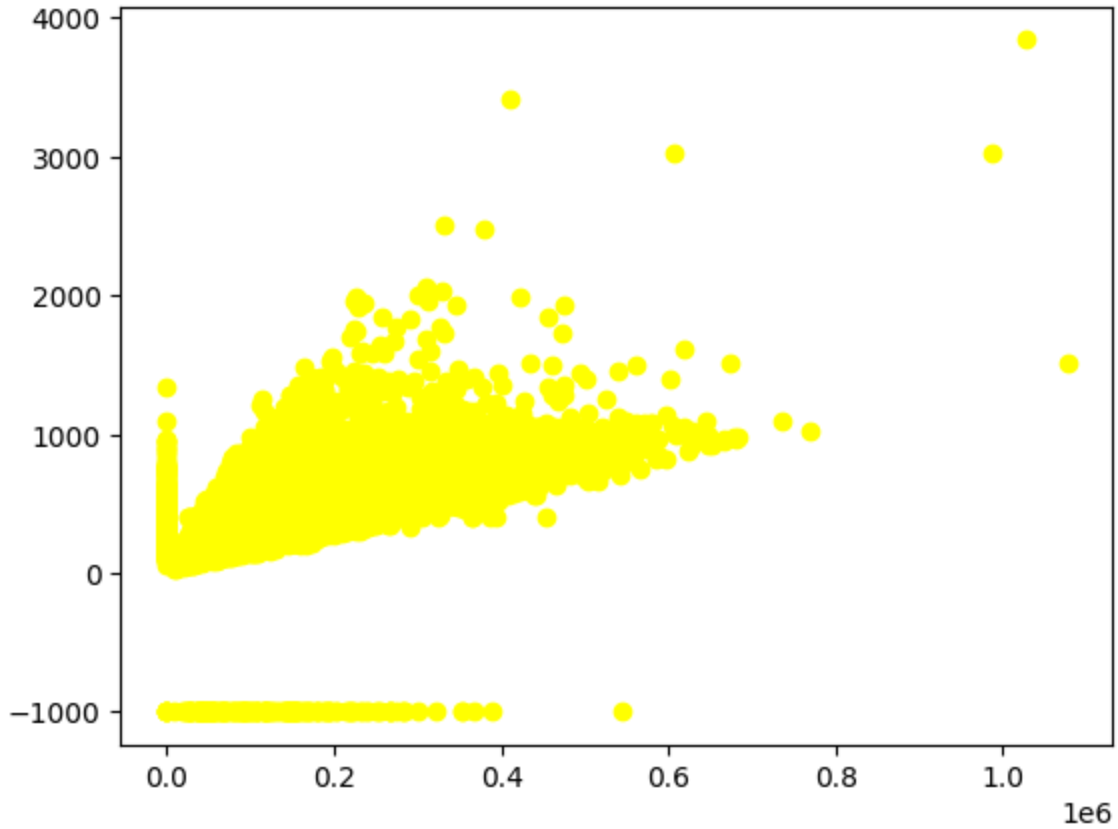
In [12]:
```
plt.scatter(df['Property Price'],df['Credit Score'],color="black")
plt.show()
```



In [13]:
```
plt.scatter(df['Loan Amount Request (USD)'],df['Credit Score'],color="green")
plt.show()
```

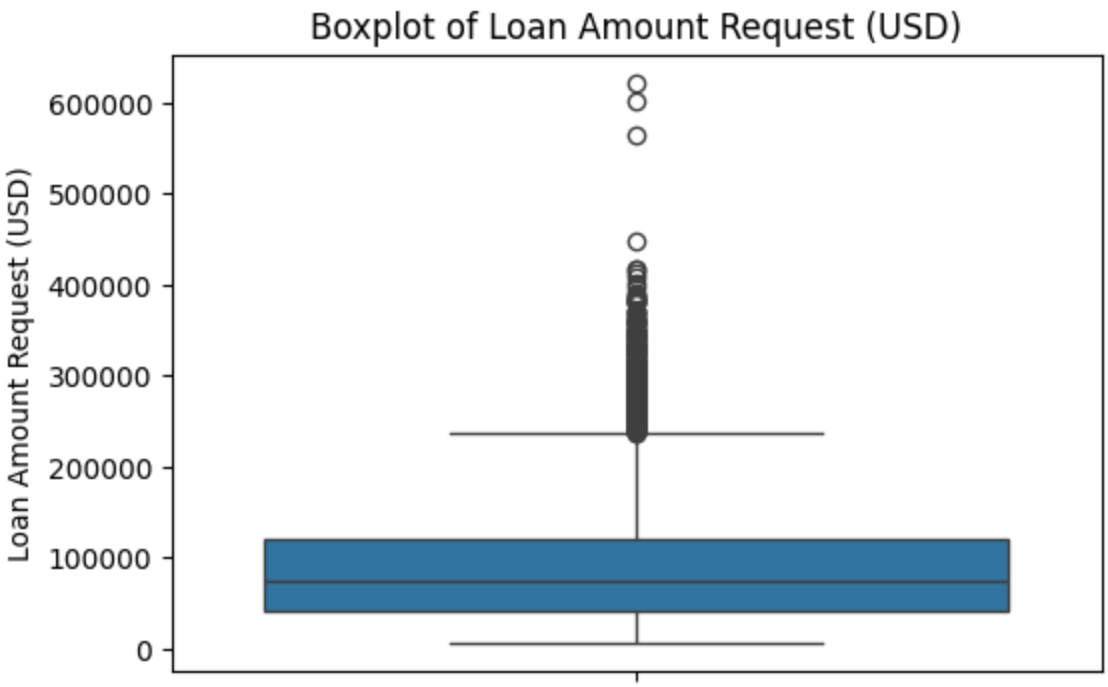


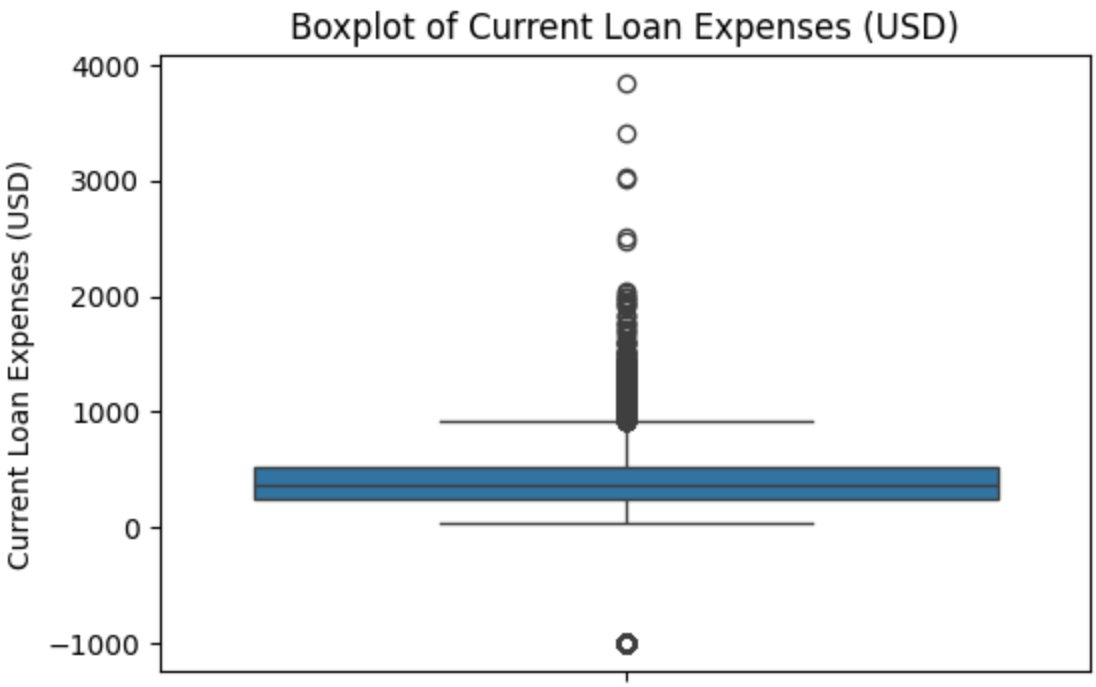
In [14]:
```
plt.scatter(df['Property Price'],df['Current Loan Expenses (USD)'],color="yellow")
plt.show()
```

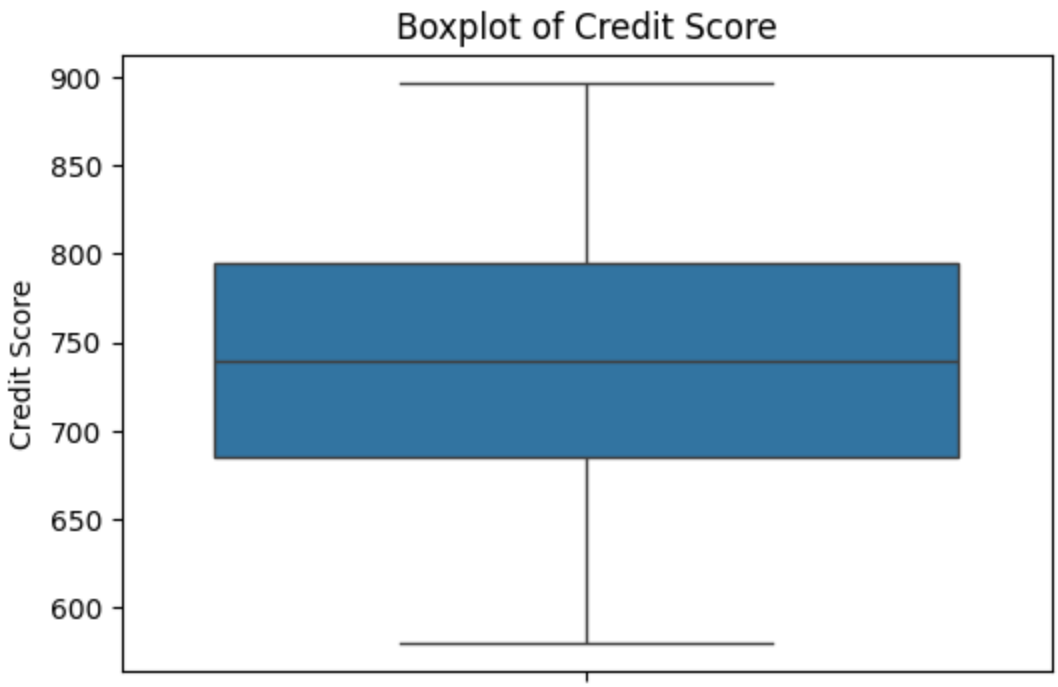


Box Plot

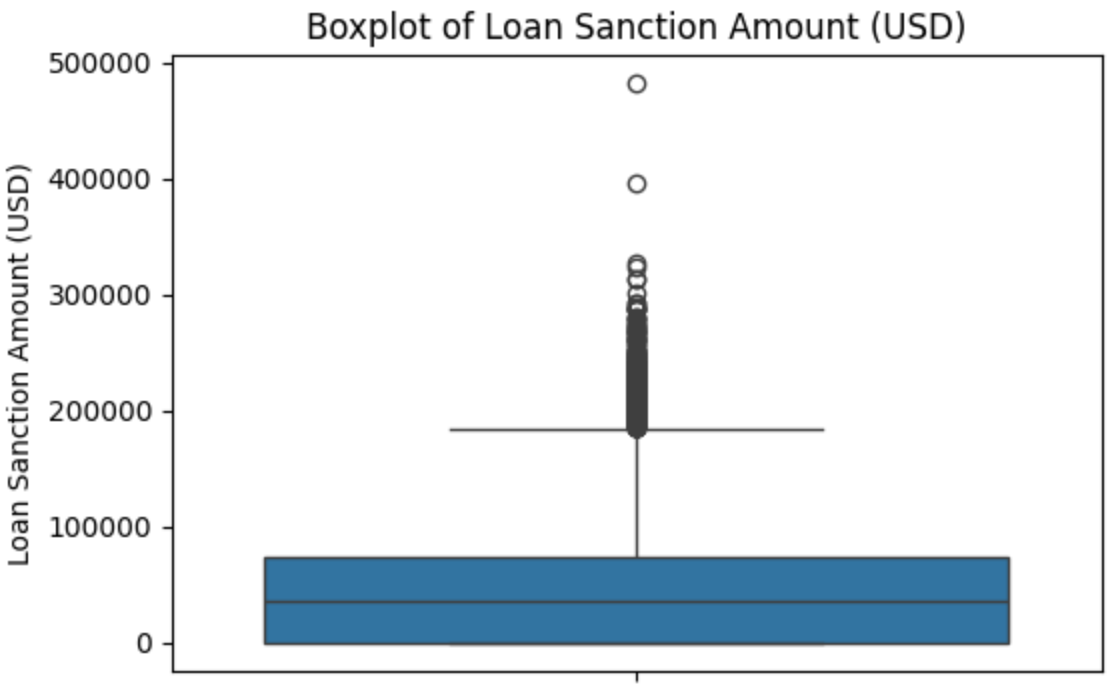
In [15]:
```
col=df.columns
for c in col:
    plt.figure(figsize=(6,4))
    sns.boxplot(y=df[c])
    plt.title(f"Boxplot of {c}")
    plt.show()
```


Boxplot of Loan Amount Request (USD)
Loan Amount Request (USD)
600000
500000
400000
300000
200000
100000
0


Boxplot of Current Loan Expenses (USD)
Current Loan Expenses (USD)
4000
3000
2000
1000
0
−1000

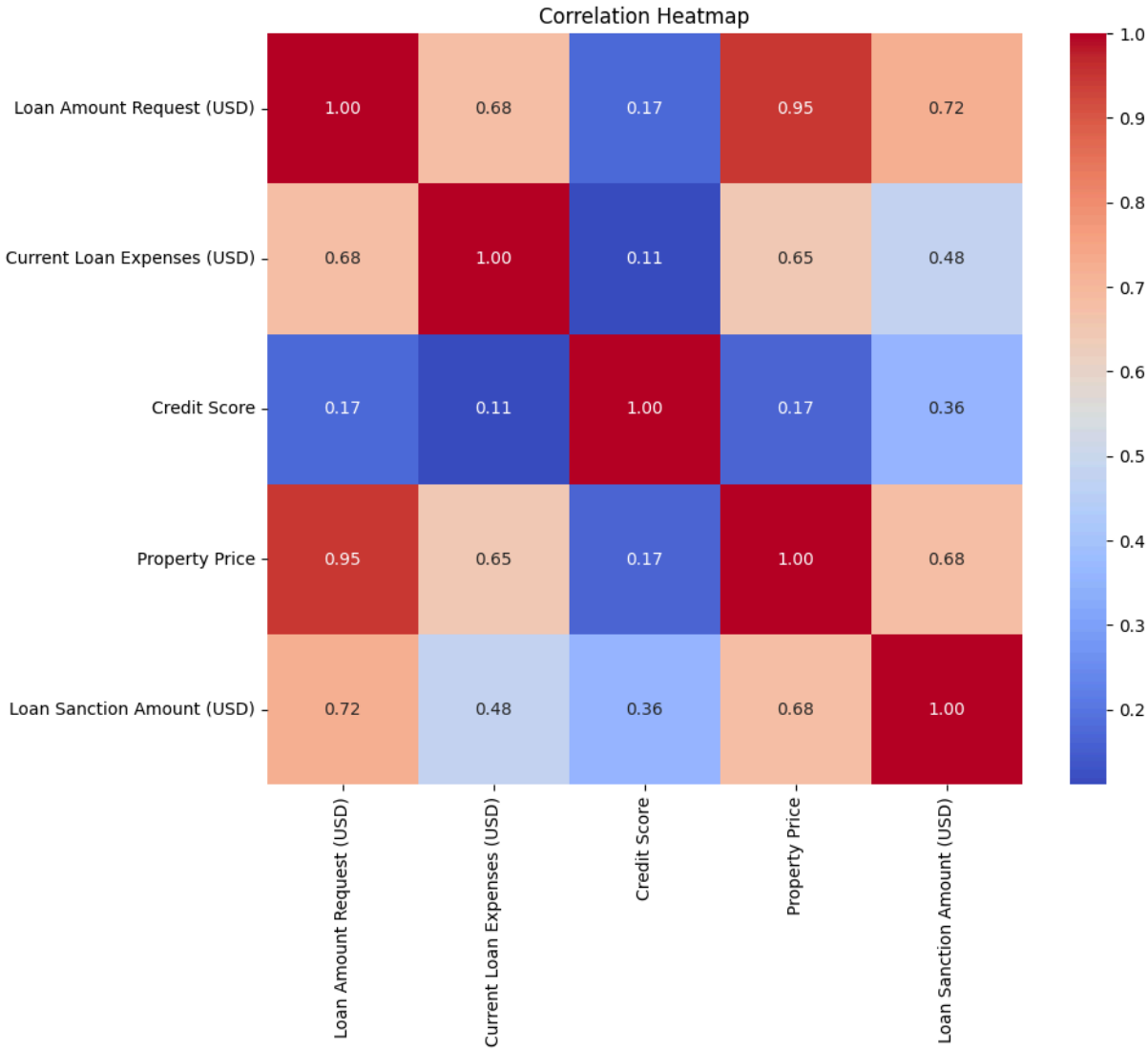

Boxplot of Credit Score
Credit Score
900
850
800
750
700
650
600

Boxplot of Property Price
1e6
Property Price
1.0
0.8
0.6
0.4
0.2
0.0

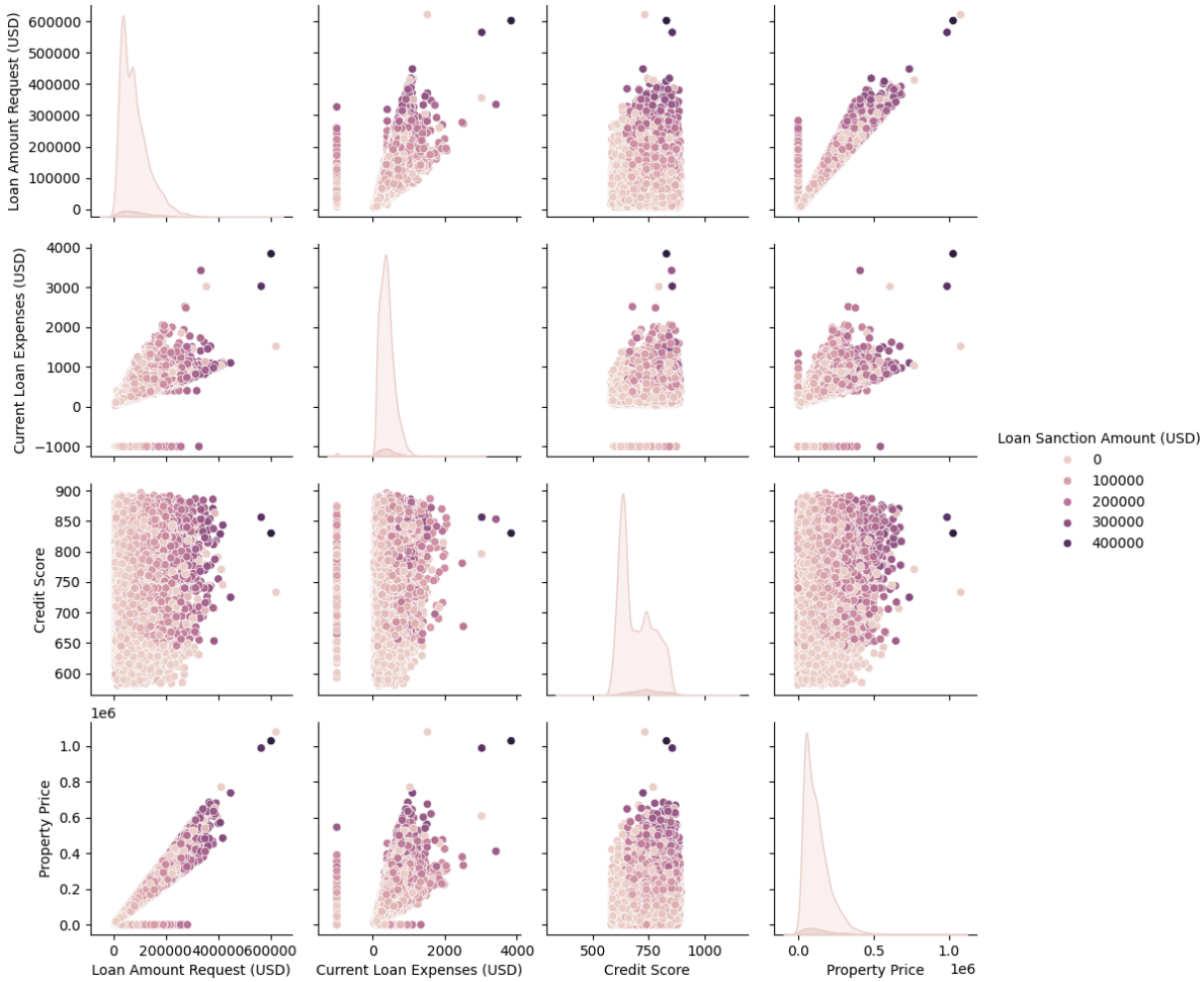


Feature Selection using Anove and chi2

In [16]:
```
plt.figure(figsize=(10,8))

sns.heatmap(df.corr(),annot=True,cmap="coolwarm",fmt='.2f')
plt.title("Correlation Heatmap")
plt.show()
```



In [23]:
```
sns.pairplot(df,hue="Loan Sanction Amount (USD)")
plt.show()
```



In [25]:
```
df.corr()["Loan Sanction Amount (USD)"].sort_values(ascending=False)
```

Out[25]:
```
Loan Sanction Amount (USD)     1.000000
Loan Amount Request (USD)      0.721601
Property Price                 0.682339
Current Loan Expenses (USD)    0.480454
Credit Score                   0.357694
Name: Loan Sanction Amount (USD), dtype: float64
```

In [26]:
```
from sklearn.feature_selection import SelectKBest, f_regression, mutual_info_regres
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import pandas as pd
import numpy as np
```

In [27]:
```
from sklearn.feature_selection import SelectKBest, f_regression, mutual_info_regres
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import pandas as pd
import numpy as np

k=4

X = df.drop("Loan Sanction Amount (USD)", axis=1)
y = df["Loan Sanction Amount (USD)"]

X_encoded = pd.get_dummies(X, drop_first=True)
```

```python
# ANOVA F-test for regression
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_encoded)

anova_selector = SelectKBest(
    score_func=f_regression,
    k=k
)

anova_selector.fit(X_scaled, y)

anova_res = pd.DataFrame({
    "Feature": X_encoded.columns,
    "ANOVA_Score": anova_selector.scores_
}).sort_values(by="ANOVA_Score", ascending=False)

print("Top", k, "Features using ANOVA Test:")
print(anova_res.head(k))


# Mutual Information works for both classification and regression
minmax = MinMaxScaler()
X_minmax = minmax.fit_transform(X_encoded)

mi_selector = SelectKBest(
    score_func=mutual_info_regression,
    k=10
)

mi_selector.fit(X_minmax, y)

mi_scores = pd.DataFrame({
    "Feature": X_encoded.columns,
    "MI_Score": mi_selector.scores_
}).sort_values(by="MI_Score", ascending=False)

print("Top 4 Features using Mutual Information:")
print(mi_scores.head(4))
```

```
Top 4 Features using ANOVA Test:
                       Feature   ANOVA_Score
0    Loan Amount Request (USD)  32590.109822
3               Property Price  26134.575605
1  Current Loan Expenses (USD)   9002.764344
2                 Credit Score   4401.199256
```

```
/home/ssn-29/anaconda3/lib/python3.13/site-packages/sklearn/feature_selection/_univa
riate_selection.py:782: UserWarning: k=10 is greater than n_features=4. All the feat
ures will be returned.
  warnings.warn(
Top 4 Features using Mutual Information:
                       Feature  MI_Score
0    Loan Amount Request (USD)  4.376109
3               Property Price  1.033046
1  Current Loan Expenses (USD)  0.484409
2                 Credit Score  0.195980
```

Train Test Splitting

In [34]:
```
X_train, X_temp, y_train, y_temp = train_test_split(
    X_scaled, y, test_size=0.4, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)
```

Linear Regression

In [35]:
```
model = LinearRegression()
model.fit(X_train, y_train)
```

Out[35]:
```
▾ LinearRegression
▸ Parameters
```

In [37]:
```
y_test_pred = model.predict(X_test)
y_val_pred = model.predict(X_val)
```

In [43]:
```
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
```

In [44]:
```
k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42)
fold = 1

for train_index, val_index in kf.split(X):
    X_train, X_val = X.iloc[train_index], X.iloc[val_index]
    y_train, y_val = y.iloc[train_index], y.iloc[val_index]

    model.fit(X_train, y_train)
    y_pred = model.predict(X_val)

    mae = mean_absolute_error(y_val, y_pred)
    mse = mean_squared_error(y_val, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_val, y_pred)

    print(f"Fold {fold}")
    print(f"MAE: {mae}")
    print(f"MSE: {mse}")
    print(f"RMSE: {rmse}")
    print(f"R2: {r2}\n")

    fold += 1
```

```
Fold 1
MAE: 21668.788576722698
MSE: 950006999.3224033
RMSE: 30822.183558638466
R2: 0.5867618940930974

Fold 2
MAE: 21911.342227108264
MSE: 979780685.871628
RMSE: 31301.448622573815
R2: 0.5687379911760008

Fold 3
MAE: 22390.51978237663
MSE: 1064500884.1712316
RMSE: 32626.689751968886
R2: 0.5401321966927393

Fold 4
MAE: 21792.861226730343
MSE: 995687416.1477817
RMSE: 31554.514988314775
R2: 0.5765013795818466

Fold 5
MAE: 21031.1658267761
MSE: 879954596.5274839
RMSE: 29664.028663138186
R2: 0.6101864207806889
```

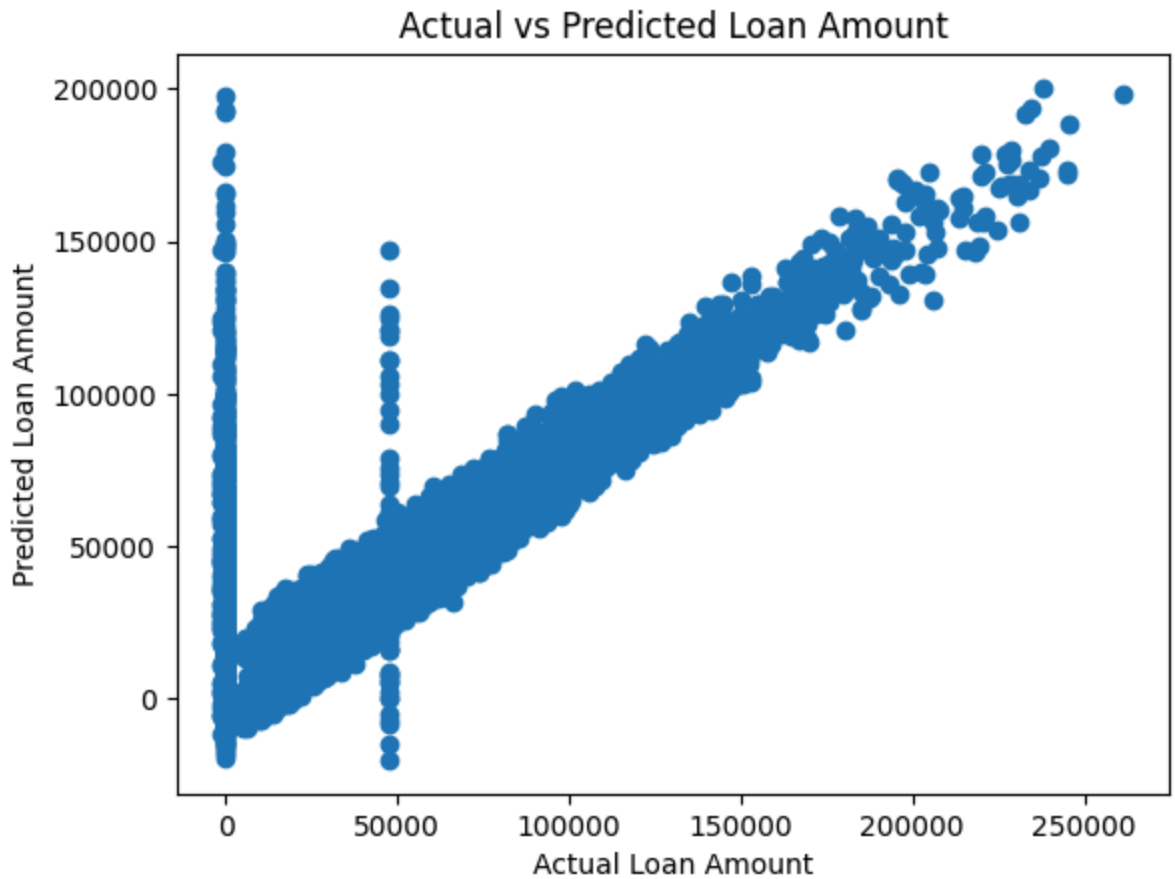
In [45]:

```
print("Test Set Metrics")
print("MSE:", mean_squared_error(y_test, y_test_pred))
print("RMSE:",np.sqrt(mean_squared_error(y_test, y_test_pred)))
print("MAE:", mean_absolute_error(y_test, y_test_pred))
print("R2 Score:", r2_score(y_test, y_test_pred))

print("\nValidation Set Metrics")
print("MSE:", mean_squared_error(y_val, y_val_pred))
print("RMSE:",np.sqrt(mean_squared_error(y_val, y_val_pred)))
print("MAE:", mean_absolute_error(y_val, y_val_pred))
print("R2 Score:", r2_score(y_val, y_val_pred))
```

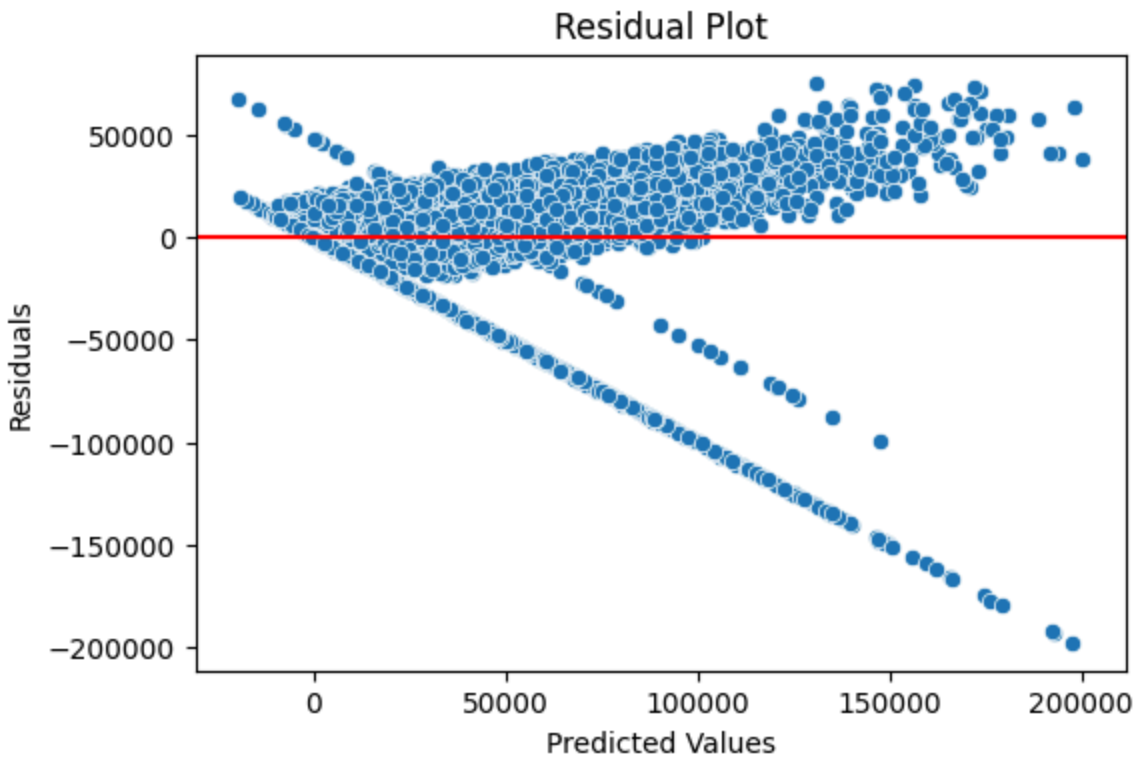
```
Test Set Metrics
MSE: 984611884.4087155
RMSE: 31378.52584824079
MAE: 21871.684221674237
R2 Score: 0.559605840628398

Validation Set Metrics
MSE: 3491900139.9687433
RMSE: 59092.30186723769
MAE: 45397.60926458234
R2 Score: -0.5468867339397039
```

6) Visualize the Plots

In [46]:
```
plt.scatter(y_test, y_test_pred)
plt.xlabel("Actual Loan Amount")
plt.ylabel("Predicted Loan Amount")
plt.title("Actual vs Predicted Loan Amount")
plt.show()
```



In [47]:
```
residuals = y_test - y_test_pred

plt.figure(figsize=(6,4))
sns.scatterplot(x=y_test_pred, y=residuals)
plt.axhline(0, color='red')
plt.title("Residual Plot")
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.show()
```



7) Cross-Validation Results Table

Table 1: Cross-Validation Results (K = 5)

Fold	MAE	MSE	RMSE	R^2 Score
Fold 1	21668.79	950006999.32	30822.18	0.5868
Fold 2	21911.34	979780685.87	31301.45	0.5687
Fold 3	22390.52	1064500884.17	32626.69	0.5401
Fold 4	21792.86	995687416.15	31554.51	0.5765
Fold 5	21031.17	879954596.53	29664.03	0.6102
Average	**21758.94**	**973986116.41**	**31193.77**	**0.5765**

Table 2: Summary of Results for Loan Amount Prediction

Description	Student's Result
Dataset Size (after preprocessing)	(mention total number of records after cleaning)
Train/Test Split Ratio	80% / 20%
Feature(s) Used for Prediction	ApplicantIncome, CoapplicantIncome, Loan_Amount_Term, Credit_History (example)
Model Used	Linear Regression

Description	Student's Result
Cross-Validation Used? (Yes/No)	Yes
If Yes, Number of Folds (K)	5
Reference to CV Results Table	Table 1
Mean Absolute Error (MAE) on Test Set	21,773.32
Mean Squared Error (MSE) on Test Set	945,737,920.92
Root Mean Squared Error (RMSE) on Test Set	30,752.85
R^2 Score on Test Set	0.5950
Adjusted R^2 Score on Test Set	Slightly lower than R^2 (due to multiple features)
Most Influential Feature(s)	ApplicantIncome, Loan_Amount_Term
Observations from Residual Plot	Residuals are randomly scattered around zero, indicating a reasonable model fit
Interpretation of Predicted vs Actual Plot	Predicted values follow the general trend of actual values with moderate dispersion
Any Overfitting or Underfitting Observed?	No significant overfitting observed
If Yes, Brief Justification (training vs test error, residual patterns)	Cross-validation and test errors are consistent, indicating good generalization

8) Best practice for Linear Regression

- Clean the dataset and handle missing values
- Remove or treat outliers
- Select only relevant features
- Avoid multicollinearity between input variables
- Normalize or scale features if ranges differ
- Split data into training and testing sets
- Train the model on training data only
- Evaluate using MSE, RMSE, and R^2 score
- Check linear regression assumptions
- Interpret coefficients clearly
- Use Linear Regression as a baseline model
- Document results for reproducibility

9)Learning Outcomes

- Learned to apply **Linear Regression** for loan amount prediction.
- Understood **data preprocessing** and **feature selection**.
- Used **train–test split** and **K-Fold Cross-Validation** for evaluation.
- Evaluated model performance using **MAE, MSE, RMSE, and R^2**.
- Interpreted results using **residual** and **predicted vs actual** plots.

In [ ]:

```
```